

| Quality Assurance in SW-Development

Assignment 3: Model-Based & Black-Box Testing

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

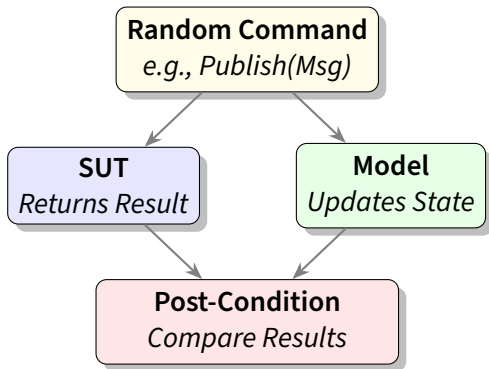
What is Model-Based Testing (MBT)?

The Concept

Instead of manually writing sequence after sequence, we use an **Abstract Model** as a source of truth.

- **SUT (System-Under-Test):** The real, complex implementation.
- **Model (State):** A simplified, high-level snapshot (e.g., simple Lists/Booleans).

We generate random sequences of *Commands* and apply them to both!



Theoretical Foundation: Hoare Logic

x^2 Hoare Triples: $\{P\} C \{Q\}$

Model-Based Testing is rooted in **Hoare Logic**, a formal system used to reason about software correctness. If a precondition is met, executing a command guarantees a specific postcondition.

In Theory:

- **P (Precondition):** Must be true *before* the command.
- **C (Command):** The program/action being executed.
- **Q (Postcondition):** Must logically be true *after* execution.

In ScalaCheck MBT:

- **P:** Generator constraints (e.g., generating only existing IDs).
- **C:** `run(sut) and nextState(state).`
- **Q:** `postCondition(state, result).`

Theoretical Foundation: Hoare Logic

$\{P\}$ Precondition

$\{Q\}$ Postcondition



Anatomy of a Command

For every action (Publish, Like, Edit...), you define a case class extending `Command`. You must implement three crucial methods:

- 1 `run(sut: SUT): Result`
Execute the command on the real `MessageBoard`.
- 2 `nextState(state: State): State`
Apply the command to your simplified model.
⚠ Rule: The model is immutable! Use `copy()`!
- 3 `postCondition(state: State, result: Try[Result]): Prop`
Check if the SUT's result matches your model's expected behavior.
(Note: 'state' here is the state BEFORE the command. Call `nextState` inside!)

Generators & Sequences

How does ScalaCheck know what commands to run? You tell it!

```
1 // Creating a Generator for a specific command
2 val genPublish: Gen[ClientMessage] = for {
3     text <- validMessageGen
4     author <- Gen.alphaStr
5 } yield new Publish(new UserMessage(author, text), -1)
6
7 // Exposing commands to the framework
8 override def genCommand(state: State): Gen[Command] = {
9     Gen.oneOf(
10         genPublish,
11         // genLike,    <-- Comment out unimplemented commands)
```

Shrinking (Finding the Bug)

🔍 The Problem

ScalaCheck generates sequences of 100+ commands. If test #87 fails, debugging the whole sequence is impossible.

🔧 The Solution

Shrinking: The framework automatically removes commands to find the *minimal* sequence that triggers the failure.

ARG_0_ORIGINAL: The full failing sequence (Huge).

ARG_0: **The shrunken, minimal failing sequence (Use this!)**

Task Breakdown

⚙️ Part 1: MBT

`MessageBoardSpecification.scala`

- Implement generators and logic for 10 commands (Publish, Like, Report, etc.).
- Validate against requirements R1-R17.
- Run with: `./gradlew testMBModel`

🏢 Part 2: Black-Box (Mutants)

Kill the Mutants!

- We provide 6 altered implementations (mutants).
- Run your MBT suite against them to find their flaws.
- `./gradlew testMutant1,`
`./gradlew testMutant2, etc.`
- We recommend to test with Gradle against every individual Mutant

Part 2 Deliverables

You need to submit three specific things for grading:

1 `MessageBoardSpecification.scala`

Your completed Model-Based Testing suite.

2 `MutantSequences.txt`

Format: `MutantNumber: ARG_0_SEQUENCE`

Example: `1: Publish(Bob, msg1), Report(Laura, Bob)`

If it's correct (no mutant): `X: NO_MUTANT`

Only the minimum failing sequence gives points!

3 `MutantDescription.md`

A brief explanation of what the bug actually was inside the mutant based on your black-box debugging.

Common Pitfalls & FAQ

⚠ Environment Issues

- **Java Version:** Use Java 11. Ensure your `JAVA_HOME` is set correctly, especially on Windows!

💡 ScalaCheck Hints

- **No Preconditions:** Don't use preconditions when testing with Pitest here. It messes up the generators. Stick to postconditions.
- **Deep Copies:** Remember, your Model state must be immutable. Use `copy()` or deep copies.

Hints for the Third Partial Exam

✓ The Teach Center Checklist

We have provided a comprehensive checklist containing all theoretical concepts that can be asked.

The Golden Rule:

If you can confidently check every box, the exam will be no problem. You must be able to recall theory, draw architectures, and calculate values!

☰ The 4 Core Pillars

- 1 MBT Basics:** Workflows, Architectures, EFAs, and taxonomies.
- 2 Combinatorial Testing:** k -way testing, calculations, and the CT Design Process.
- 3 Monkey & Fuzz Testing:** Dumb vs. Smart Monkeys, Fuzzing types (Gen/Mut), and the ATMS Architecture.
- 4 Stateful Frameworks:** Hoare Logic, Pre/Postconditions, Immutability, and Shrinking (ARG_0).

Hints for the Third Partial Exam

Study Tip:

Use the checklist to quiz each other in your groups.
If you can clearly explain the concept,
calculate examples, and draw the diagrams, you are ready!

| Quality Assurance in SW-Development

Assignment 3: Model-Based & Black-Box Testing

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security